

Optimisation

MIASHS M1 - Semestre 1

Table des matières

1. Conditions d'existence d'un minimum	1
2. Fonctions d'optimisation en Machine Learning	7
3. Algorithmes de descente de gradient	16

1. Conditions d'existence d'un minimum

1.1. Introduction

L'étude des conditions d'existence d'un minimum est fondamentale en optimisation. Nous allons examiner différentes conditions, certaines nécessaires, d'autres suffisantes, même si elles ne sont pas toujours utilisées en pratique.

1.2. Définitions préliminaires

Définition 1.1 (Minimum local)

Soit $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$. On dit que $x^* \in \Omega$ est un **minimum local** de f s'il existe $\varepsilon > 0$ tel que :

$$f(x^*) \leq f(x), \quad \forall x \in B(x^*, \varepsilon) \cap \Omega$$

où $B(x^*, \varepsilon) = \{x \in \mathbb{R}^n : \|x - x^*\| < \varepsilon\}$.

Définition 1.2 (Minimum global)

On dit que $x^* \in \Omega$ est un **minimum global** de f si :

$$f(x^*) \leq f(x), \quad \forall x \in \Omega$$

1.3. Conditions nécessaires

1.3.1. Condition du premier ordre

Théorème 1.3 (CNO - Condition nécessaire)

Si $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$ est différentiable et si x^* est un minimum local de f dans l'intérieur de Ω , alors :

$$\nabla f(x^*) = 0$$

Remarque : Cette condition est nécessaire mais **pas suffisante**. Un point vérifiant $\nabla f(x^*) = 0$ est appelé **point critique** ou **point stationnaire**.

1.3.2. Condition du second ordre (nécessaire)

Théorème 1.4 (CNSO)

Si $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$ est deux fois différentiable et si x^* est un minimum local, alors :

1. $\nabla f(x^*) = 0$

2. La matrice hessienne $H_f(x^*)$ est **semi-définie positive** :

$$v^\top H_f(x^*) v \geq 0, \quad \forall v \in \mathbb{R}^n$$

Remarque importante : Cette condition est nécessaire mais pas suffisante pour garantir l'existence d'un minimum local.

1.4. Conditions suffisantes

1.4.1. Condition du second ordre (suffisante)

Théorème 1.5 (CSSO)

Si $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$ est deux fois différentiable et si :

1. $\nabla f(x^*) = 0$
2. $H_f(x^*)$ est **définie positive** :

$$v^\top H_f(x^*)v > 0, \quad \forall v \neq 0$$

Alors x^* est un **minimum local strict**.

1.4.2. Convexité et minimum global

Définition 1.6 (Fonction convexe)

Une fonction $f : \Omega \rightarrow \mathbb{R}$ est dite **convexe** sur un ensemble convexe Ω si :

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

pour tout $x, y \in \Omega$ et $\lambda \in [0, 1]$.

Théorème 1.7 (Convexité et optimalité)

Si f est convexe et différentiable sur Ω convexe, alors :

$$\nabla f(x^*) = 0 \Rightarrow x^* \text{ est un minimum global}$$

Remarque cruciale : Une fonction peut avoir un minimum global **sans être convexe** sur son domaine. La convexité est une condition **suffisante** mais **pas nécessaire**.

1.5. Contre-exemples et cas limites

1.5.1. Minimum sans convexité

Exemple 1.8

Considérons $f(x) = x^4 - x^2$ sur $\mathbb{R}$.

- $f'(x) = 4x^3 - 2x = 0 \Rightarrow x \in \left\{0, \pm \frac{1}{\sqrt{2}}\right\}$
- f n'est **pas convexe** sur $\mathbb{R}$ (car $f''(0) = -2 < 0$)
- Pourtant, $x = \pm \frac{1}{\sqrt{2}}$ sont des **minima globaux**

Exercice

Démontrez les observations issues de l'exemple précédent

1.5.2. Point critique sans minimum

Exemple 1.9

Soit $f(x) = x^3$ sur $\mathbb{R}$.

- $f'(0) = 0$: $x = 0$ est un point critique
- $f''(0) = 0$: condition du second ordre **non concluante**
- $x = 0$ n'est **ni un minimum ni un maximum** (point d'inflexion)

Exercice

Démontrez les observations issues de l'exemple précédent

1.5.3. Hessienne semi-définie positive sans minimum

Exemple 1.10

Soit $f(x, y) = x^2 - x^4 - y^4$ sur $\mathbb{R}^2$.

Au point $(0, 0)$:

- $\nabla f(0, 0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
- $H_f(0, 0) = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$ est **semi-définie positive**
- Mais $(0, 0)$ n'est **pas un minimum local** (c'est un point selle)

Exercice

Démontrez les observations issues de l'exemple précédent

1.6. Coercivité et existence d'un minimum global

Définition 1.11 (Fonction coercive)

Une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est **coercive** si :

$$\lim_{\|x\| \rightarrow +\infty} f(x) = +\infty$$

Théorème 1.12 (Existence d'un minimum global)

Si $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est continue et coercive, alors f admet **au moins un minimum global**.

Démonstration (idée) :

La coercivité implique que l'ensemble de sous-niveau

$$S_c = \{x : f(x) \leq c\}$$

est borné pour tout $c \in \mathbb{R}$.

Comme f est continue, S_c est également fermé, donc compact (par le théorème de Heine-Borel).

Par le **théorème des bornes atteintes** (ou théorème de Weierstrass), toute fonction continue sur un compact atteint ses bornes. Ainsi, f atteint son minimum sur S_c , et donc sur $\mathbb{R}^n$ tout entier.

1.7. Synthèse : hiérarchie des conditions

Condition	Type	Garantit
$\nabla f(x^*) = 0$	Nécessaire	Point critique
$H_f(x^*) \geq 0$	Nécessaire	Candidat minimum
$H_f(x^*) > 0$	Suffisante	Minimum local
f convexe + $\nabla f(x^*) = 0$	Suffisante	Minimum global
f coercive + continue	Suffisante	Existence minimum

(Attention à l'abus de notation avec la Hessienne !!!)

Conclusion : En pratique, on cherche souvent des conditions suffisantes (convexité, coercivité) pour garantir l'optimalité. Cependant, de nombreux problèmes réels ne satisfont pas ces conditions et possèdent néanmoins des solutions optimales.

2. Fonctions d'optimisation en Machine Learning

2.1. Introduction

En apprentissage automatique, les fonctions objectifs à minimiser possèdent une structure particulière qui facilite l'analyse et l'optimisation.

La forme générale est :

$$f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$$

où :

- $x \in \mathbb{R}^d$ est le vecteur de paramètres du modèle
- n est le nombre d'exemples d'entraînement
- $f_i(x)$ représente la perte sur l'exemple i

2.2. Propriétés de conservation

2.2.1. Conservation de la convexité

Théorème 2.1 (Somme de fonctions convexes)

Si $f_1, f_2, \dots, f_n$ sont des fonctions convexes sur Ω convexe, alors :

$$f(x) = \sum_{i=1}^n \alpha_i f_i(x)$$

est convexe pour tout $\alpha_i \geq 0$.

En particulier, la moyenne $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$ est convexe.

Démonstration :

Soient $x, y \in \Omega$ et $\lambda \in [0, 1]$. Calculons :

$$\begin{aligned} f(\lambda x + (1 - \lambda)y) &= \sum_{i=1}^n \alpha_i f_i(\lambda x + (1 - \lambda)y) \\ &\leq \sum_{i=1}^n \alpha_i [\lambda f_i(x) + (1 - \lambda)f_i(y)] \quad \text{par conv de chaque} \\ &= \sum_{i=1}^n [\alpha_i \lambda f_i(x) + \alpha_i (1 - \lambda)f_i(y)] \\ &= \lambda \sum_{i=1}^n \alpha_i f_i(x) + (1 - \lambda) \sum_{i=1}^n \alpha_i f_i(y) \\ &= \lambda f(x) + (1 - \lambda)f(y) \end{aligned}$$

Donc f est convexe.

Exercice

Démontrez le théorème précédent en supposant chaque f_i de classe $\mathcal{C}^2$ avec un argument sur la Hessienne ou sur la différentielle seconde.

2.2.2. Conservation de la différentiabilité

Proposition 2.2 (Gradient d'une somme)

Si chaque f_i est différentiable, alors :

$$\nabla f(x) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(x)$$

Cette propriété est fondamentale pour les algorithmes de descente de gradient.

Proposition 2.3 (Hessienne d'une somme)

Si chaque f_i est deux fois différentiable, alors :

$$H_f(x) = \frac{1}{n} \sum_{i=1}^n H_{f_i}(x)$$

2.2.3. Conservation de la coercivité

Proposition 2.4 (Somme et coercivité)

Si au moins une fonction f_i est coercive et les autres sont minorées, alors $f = \frac{1}{n} \sum_{i=1}^n f_i$ est coercive.

Remarque : En pratique, on ajoute souvent un terme de régularisation coercif pour garantir l'existence d'un minimum.

2.2.4. Conservation de la convexité par composition affine

Théorème 2.5 (Composition affine et convexité)

Soit $g : \mathbb{R}^m \rightarrow \mathbb{R}$ une fonction convexe et $A : \mathbb{R}^n \rightarrow \mathbb{R}^m$ une fonction affine définie par :

$$A(x) = Mx + b$$

où $M \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$.

Alors la composée $f(x) = g(A(x)) = g(Mx + b)$ est convexe sur $\mathbb{R}^n$.

Démonstration :

Soient $x, y \in \mathbb{R}^n$ et $\lambda \in [0, 1]$. Il faut montrer que :

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

Calculons :

$$\begin{aligned} f(\lambda x + (1 - \lambda)y) &= g(A(\lambda x + (1 - \lambda)y)) \\ &= g(M(\lambda x + (1 - \lambda)y) + b) \\ &= g(\lambda Mx + (1 - \lambda)My + b) \\ &= g(\lambda(Mx + b) + (1 - \lambda)(My + b)) \\ &= g(\lambda A(x) + (1 - \lambda)A(y)) \end{aligned}$$

Par convexité de g :

$$\begin{aligned} g(\lambda A(x) + (1 - \lambda)A(y)) &\leq \lambda g(A(x)) + (1 - \lambda)g(A(y)) \\ &= \lambda f(x) + (1 - \lambda)f(y) \end{aligned}$$

Donc f est convexe.

2.2.5. Applications importantes

Exemple 2.5.1 (Norme de vecteurs transformés)

Pour $p \geq 1$, la fonction $f(x) = \|Mx + b\|_p$ est convexe car :

- $A(x) = Mx + b$ est affine
- $g(u) = \|u\|_p$ est convexe

Exemple 2.5.2 (Support Vector Machine)

La fonction de perte hinge transformée :

$$f(w) = \sum_{i=1}^n \max(0, 1 - y_i(w^\top x_i + b))$$

est convexe car :

- $A_{i(w)} = y_i w^\top x_i$ est affine en w
- $g(t) = \max(0, 1 - t)$ est convexe

2.3. Exemples en Machine Learning

2.3.1. Régression linéaire

Exemple 2.6 (Moindres carrés)

Pour des données $(x_i, y_i)_{i=1}^n$, la fonction de perte est :

$$f(w) = \frac{1}{2n} \sum_{i=1}^n (y_i - w^\top x_i)^2$$

- Chaque $f_i(w) = \frac{1}{2}(y_i - w^\top x_i)^2$ est **convexe**
- Donc f est **convexe**
- $\nabla f(w) = -\frac{1}{n} \sum_{i=1}^n (y_i - w^\top x_i) x_i = \frac{1}{n} (X^\top X w - X^\top y)$
- $H_f(w) = \frac{1}{n} \sum_{i=1}^n x_i x_i^\top = \frac{1}{n} X^\top X$

Proposition 2.6.1 (Solution en forme close)

Si $X^\top X$ est **inversible**, alors la minimisation de f admet une **solution unique en forme close** :

$$w^* = (X^\top X)^{-1} X^\top y$$

obtenue en résolvant $\nabla f(w) = 0$.

Remarque : $X^\top X$ est inversible si et seulement si $\text{rang}(X) = d$, i.e. les colonnes de X sont linéairement indépendantes.

Proposition 2.6.2 (Régularisation Ridge)

Considérons la fonction régularisée :

$$f_\lambda(w) = \frac{1}{2n} \sum_{i=1}^n (y_i - w^\top x_i)^2 + \frac{\lambda}{2} \|w\|_2^2$$

avec $\lambda > 0$. Alors :

1. f_λ est **strictement convexe**
2. $H_{f_\lambda}(w) = \frac{1}{n} X^\top X + \lambda I_d$ est **définie positive**
3. Le minimum admet **toujours** une solution unique :

$$w_\lambda^* = (X^\top X + n\lambda I_d)^{-1} X^\top y$$

Démonstration (idée) :

- La hessienne $H_{f_\lambda}(w) = \frac{1}{n} X^\top X + \lambda I_d$ vérifie :

$$v^\top H_{f_\lambda}(w) v = \frac{1}{n} v^\top X^\top X v + \lambda \|v\|^2 \geq \lambda \|v\|^2 > 0$$

pour tout $v \neq 0$. Donc f_λ est strictement convexe.

- La matrice $X^\top X + n\lambda I_d$ est définie positive, donc **toujours inversible** pour $\lambda > 0$.

Exercice

1. Montrez que $w_\lambda^* \rightarrow w^*$ quand $\lambda \rightarrow 0$ (si $X^\top X$ inversible)
2. Calculez $\lim_{\lambda \rightarrow +\infty} w_\lambda^*$
3. Interprétez géométriquement l'effet de λ sur la solution

2.3.2. Régression logistique

Exemple 2.7 (Log-perte)

Pour des données $(x_i, y_i)_{i=1}^n$ avec $y_i \in \{0, 1\}$:

$$f(w) = \frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-y_i w^\top x_i))$$

- Chaque f_i est **convexe** (composition de fonctions convexes)
- Donc f est **convexe**
- Pas de forme fermée pour le minimum, mais garantie d'unicité

2.3.3. Régularisation

Exemple 2.8 (Ridge et Lasso)

On ajoute souvent un terme de régularisation :

$$f(w) = \frac{1}{n} \sum_{i=1}^n \ell_i(w) + \lambda r(w)$$

où :

- **Ridge** : $r(w) = \|w\|_2^2$ (convexe, différentiable, coercif)

- **Lasso** : $r(w) = \|w\|_1$ (convexe, non différentiable partout)

2.4. Non-convexité en Deep Learning

Attention

Pour les réseaux de neurones, même si chaque f_i provient d'une architecture identique et même si la fonction de perte est convexe, la fonction totale :

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_{\theta}(x_i), y_i)$$

est généralement **non convexe** à cause de la composition non linéaire des couches.

Conséquence : Les algorithmes d'optimisation ne garantissent que des minima locaux, pas globaux. On notera cependant que l'exploration dans l'espace des paramètres en grande dimension (million ou milliards de paramètres) a tendance à garantir la découverte de minimum globaux ou à minima ayant de bonnes propriétés de généralisation.

2.5. Synthèse des propriétés

Propriété	Conservée par somme	Exemple ML
Convexité	✓ Oui	Régression linéaire
Différentiabilité	✓ Oui	Log-perte
Coercivité	✓ Si ≥ 1 terme coercif	Ridge

Lipschitz-continuité	✓ Oui	Gradient borné
Forte convexité	✓ Oui	Ridge avec $\lambda > 0$

Conclusion : La structure de somme des fonctions en ML permet de transférer les propriétés individuelles à la fonction globale, facilitant l'analyse théorique et la conception d'algorithmes d'optimisation efficaces.

Exercice

Montrez que si chaque f_i est μ -fortement convexe, alors $f = \frac{1}{n} \sum_{i=1}^n f_i$ est $n\mu$ -fortement convexe.

Exercice

Montrez que si chaque gradient ∇f_i est L_i -Lipschitz continu, alors $\nabla f = \frac{1}{n} \sum_{i=1}^n \nabla f_i$ est L -Lipschitz continu avec $L = \frac{1}{n} \sum_{i=1}^n L_i$.

Rappel : Un gradient ∇g est L -Lipschitz continu si :

$$\|\nabla g(x) - \nabla g(y)\| \leq L \|x - y\|, \quad \forall x, y \in \mathbb{R}^d$$

3. Algorithmes de descente de gradient

3.1. Introduction

Les algorithmes de descente de gradient sont les méthodes les plus utilisées en optimisation, notamment en machine learning. L'idée fondamentale est d'itérer dans la direction opposée au gradient pour diminuer la valeur de la fonction.

3.2. Descente de gradient classique

3.2.1. Principe général

Algorithme 3.1 (Descente de gradient)

Soit $f : \mathbb{R}^d \rightarrow \mathbb{R}$ différentiable. À partir d'un point initial $x^{(0)}$, on itère :

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$$

où $\alpha_k > 0$ est le **pas d'apprentissage** (learning rate) à l'itération k .

Remarque : Le pas d'apprentissage peut être constant ou évoluer selon une politique particulière. On parle de *learning rate scheduler*.

3.2.2. Intuition géométrique

Question : Pourquoi se déplacer dans la direction $-\nabla f(x)$ est-il une bonne idée ?

Proposition 3.2 (Direction de plus forte descente)

Le gradient $-\nabla f(x)$ est la direction de **plus forte descente** de f en x , i.e., parmi toutes les directions unitaires d avec $\|d\| = 1$, celle qui minimise la dérivée directionnelle :

$$d^* = \arg \min_{\|d\|=1} \nabla f(x)^\top d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$$

Démonstration :

Pour minimiser la dérivée directionnelle $\nabla f(x)^\top d$ sous la contrainte $\|d\| = 1$:

$$\nabla f(x)^\top d = \|\nabla f(x)\| \cdot \|d\| \cdot \cos(\theta) = \|\nabla f(x)\| \cos(\theta)$$

où θ est l'angle entre $\nabla f(x)$ et d .

Le minimum est atteint pour $\cos(\theta) = -1$, c'est-à-dire $\theta = \pi$, soit :

$$d^* = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$$

3.2.3. Justification par développement limité

Proposition 3.3 (Décroissance locale)

Si f est $\mathcal{C}^1$ et $\nabla f(x) \neq 0$, alors pour α suffisamment petit :

$$f(x - \alpha \nabla f(x)) < f(x)$$

Démonstration :

Par développement de Taylor à l'ordre 1 :

$$\begin{aligned} f(x - \alpha \nabla f(x)) &= f(x) + \nabla f(x)^\top (-\alpha \nabla f(x)) + o(\alpha \|\nabla f(x)\|) \\ &= f(x) - \alpha \|\nabla f(x)\|^2 + o(\alpha) \end{aligned}$$

Pour α assez petit, le terme $-\alpha \|\nabla f(x)\|^2$ domine, donc :

$$f(x - \alpha \nabla f(x)) < f(x)$$

Remarque cruciale : Cette décroissance n'est garantie que **localement**, pour un pas α suffisamment petit. Ce **n'est pas**, non plus, une preuve de convergence.

3.2.4. Choix du pas d'apprentissage

Cas 1 : Pas constant

$\alpha_k = \alpha$ pour tout k .

- Simple à implémenter
- Nécessite un réglage délicat : trop grand $\rightarrow$ divergence, trop petit $\rightarrow$ convergence lente

Cas 2 : Recherche linéaire (line search)

À chaque itération, on choisit α_k qui minimise :

$$\alpha_k = \arg \min_{\alpha > 0} f(x^{(k)} - \alpha \nabla f(x^{(k)}))$$

- Garantit une décroissance maximale à chaque itération
- Coûteux en calcul

Cas 3 : Pas décroissant

$$\alpha_k = \frac{\alpha_0}{k} \text{ ou } \alpha_k = \frac{\alpha_0}{\sqrt{k}}$$

- Garantit la convergence sous certaines conditions
- Utilisé en pratique avec des variantes adaptatives

Les bibliothèques de *deep learning* fournissent une grande variété de *learning rate scheduler*.

3.2.5. Convergence pour les fonctions convexes

Théorème 3.4 (Convergence - cas convexe)

Supposons que f est convexe, L -lisse (gradient L -Lipschitz) et admet un minimum x^* .

Si on choisit $\alpha_k = \frac{1}{L}$ (pas constant), alors :

$$f(x^{(k)}) - f(x^*) \leq \frac{L\|x^{(0)} - x^*\|^2}{2k}$$

Convergence en $\mathcal{O}(\frac{1}{k})$.

Remarque : Pour les fonctions **fortement convexes**, on obtient une convergence **exponentielle** en $\mathcal{O}(e^{-\mu \frac{k}{L}})$.

3.2.6. Exemple : régression linéaire

Exemple 3.5 (Descente de gradient pour les moindres carrés)

Pour $f(w) = \frac{1}{2n} \|y - Xw\|^2$, l'algorithme devient :

$$\begin{aligned} w^{(k+1)} &= w^{(k)} - \alpha \nabla f(w^{(k)}) \\ &= w^{(k)} - \frac{\alpha}{n} X^\top (Xw^{(k)} - y) \\ &= w^{(k)} - \frac{\alpha}{n} \sum_{i=1}^n (w^{(k)\top} x_i - y_i) x_i \end{aligned}$$

Coût par itération : $\mathcal{O}(nd)$ où n = nombre d'exemples, d = dimension.

3.3. Momentum et méthodes accélérées

3.3.1. Gradient avec momentum

Algorithme 3.6 (Gradient avec momentum)

On maintient une **vitesse** $v^{(k)}$ qui accumule les gradients passés :

$$v^{(k+1)} = \beta v^{(k)} + \nabla f(x^{(k)})$$

$$x^{(k+1)} = x^{(k)} - \alpha v^{(k+1)}$$

où $\beta \in [0, 1)$ est le **coefficient de momentum** (typiquement $\beta = 0.9$).

Intuition : Le momentum permet d'**accélérer** dans les directions cohérentes et d'**amortir** les oscillations.

Remarque 3.7 (Développement de la vitesse)

En déroulant la récurrence :

$$v^{(k)} = \sum_{i=0}^{k-1} \beta^i \nabla f(x^{(k-1-i)})$$

Le momentum donne plus de poids aux gradients récents (décroissance exponentielle).

Attention, dans la formule précédente le i de β^i est bien un exposant...

3.3.2. Méthode de Nesterov

Algorithme 3.8 (Nesterov Accelerated Gradient)

Variante « anticipative » du momentum :

$$\tilde{x}^{(k)} = x^{(k)} + \beta(x^{(k)} - x^{(k-1)})$$

$$x^{(k+1)} = \tilde{x}^{(k)} - \alpha \nabla f(\tilde{x}^{(k)})$$

On évalue le gradient au point « anticipé » $\tilde{x}^{(k)}$.

Avantage théorique : Convergence en $\mathcal{O}(\frac{1}{k^2})$ pour les fonctions convexes (contre $\mathcal{O}(\frac{1}{k})$ sans accélération).

3.4. Descente de gradient stochastique (SGD)

3.4.1. Principe

En machine learning, calculer le gradient exact

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w)$$

est **coûteux** pour n grand.

Algorithme 3.9 (Stochastic Gradient Descent)

À chaque itération k :

1. Tirer aléatoirement un indice $i_k \in \{1, \dots, n\}$
2. Mettre à jour :

$$w^{(k+1)} = w^{(k)} - \alpha_k \nabla f_{i_k}(w^{(k)})$$

On utilise le gradient d'un **seul exemple** au lieu de la moyenne.

Coût par itération : $\mathcal{O}(d)$ au lieu de $\mathcal{O}(nd)$ $\rightarrow$ gain facteur n !

3.4.2. Gradient stochastique non biaisé

Proposition 3.10 (Gradient non biaisé)

Si i_k est tiré uniformément dans $\{1, \dots, n\}$, alors :

$$\mathbb{E}[\nabla f_{i_k}(w)] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w) = \nabla f(w)$$

Le gradient stochastique est un **estimateur non biaisé** du gradient exact.

Démonstration :

$$\mathbb{E}_{i_k} [\nabla f_{i_k}(w)] = \sum_{i=1}^n \mathbb{P}(i_k = i) \nabla f_i(w) = \sum_{i=1}^n \frac{1}{n} \nabla f_i(w) = \nabla f(w)$$

3.4.3. Variance du gradient stochastique

Attention : variance élevée

Bien que non biaisé, le gradient stochastique a une **variance** :

$$\mathbb{E}[\|\nabla f_{i_k}(w) - \nabla f(w)\|^2]$$

Cette variance peut être grande si les gradients individuels sont très différents.

Conséquence : Les itérés $w^{(k)}$ oscillent autour du minimum au lieu de converger exactement.

Proposition 3.11 (Convergence SGD)

Pour une fonction convexe et L -lisse, avec un pas décroissant $\alpha_k = \frac{\alpha}{\sqrt{k}}$:

$$\mathbb{E}[f(\bar{w}_K)] - f(w^*) = \mathcal{O}\left(\frac{1}{\sqrt{K}}\right)$$

où $\bar{w}_K = \frac{1}{K} \sum_{k=1}^K w^{(k)}$ est la moyenne des itérés.

3.4.4. Mini-batch gradient descent

Algorithme 3.12 (Mini-batch SGD)

Compromis entre gradient complet et stochastique :

1. Tirer aléatoirement un sous-ensemble $\mathcal{B}_k \subset \{1, \dots, n\}$ de taille $|\mathcal{B}_k| = b$ (batch size)
2. Mettre à jour :

$$w^{(k+1)} = w^{(k)} - \alpha_k \frac{1}{b} \sum_{i \in \mathcal{B}_k} \nabla f_i(w^{(k)})$$

Avantages :

- Réduit la variance : $\text{Var} \propto \frac{1}{b}$
- Parallélisation possible (GPU)
- Taille typique en *deep learning* : $b \in \{32, 64, 128, 256\}$

3.4.5. Comparaison des vitesses de convergence

Remarque 3.13 (En espérance vs variance)

Pour un mini-batch de taille b :

$$\mathbb{E}[\tilde{\nabla} f(w)] = \nabla f(w) \quad \text{non biaisé}$$

$$\text{Var}[\tilde{\nabla} f(w)] = \frac{\sigma^2}{b} \quad \text{variance réduite}$$

où $\sigma^2 = \mathbb{E}[\|\nabla f_i(w) - \nabla f(w)\|^2]$ est la variance individuelle.

Méthode	Coût/itération	Convergence
GD	$\mathcal{O}(nd)$	$\mathcal{O}\left(\frac{1}{k}\right)$
SGD	$\mathcal{O}(d)$	$\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$
Mini-batch (b)	$\mathcal{O}(bd)$	$\mathcal{O}\left(\frac{1}{\sqrt{bk}}\right)$

En pratique : SGD et mini-batch sont préférés pour les grands datasets car le gain en vitesse par itération compense la convergence plus lente.

Exercice

1. Montrez que pour un mini-batch de taille b , la variance du gradient estimé est $\frac{\sigma^2}{b}$.
2. Estimez le nombre d'itérations nécessaires pour que mini-batch SGD ($b = 32$) fasse le même nombre d'évaluations de gradient qu'une itération de gradient complet sur $n = 10,000$ exemples.
3. Implémentez GD et SGD pour la régression linéaire en deux dimensions et comparez les trajectoires en affichant les itérés au fur et à mesure de l'apprentissage.

3.5. Descente par coordonnées (Coordinate Descent)

3.5.1. Principe général

Au lieu de minimiser selon toutes les variables simultanément, on minimise selon **une coordonnée à la fois** en fixant les autres.

Algorithme 3.13 (Descente par coordonnées - version cyclique)

Soit $f : \mathbb{R}^d \rightarrow \mathbb{R}$ différentiable. À partir d'un point initial $x^{(0)} \in \mathbb{R}^d$:

Pour $k = 0, 1, 2, \dots$:

Pour $i = 1, \dots, d$:

 Minimiser selon la coordonnée i en utilisant les valeurs déjà mises à jour :

$$x_i^{(k+1)} \leftarrow \arg \min_{t \in \mathbb{R}} f\left(x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}, t, x_{i+1}^{(k)}, \dots, x_d^{(k)}\right)$$

Attention : utilisation immédiate des valeurs mises à jour

Lorsqu'on met à jour x_i , on utilise :

- Les **nouvelles** valeurs $x_1, \dots, x_{i-1}$ (déjà mises à jour)
- Les **anciennes** valeurs $x_{i+1}, \dots, x_d$ (pas encore traitées)

C'est ce qui différencie coordinate descent de la simple application du gradient sur chaque coordonnée !

3.5.2. Comparaison : CD vs Gradient par coordonnées (Optionnel)

Différence cruciale

Descente par coordonnées (Gauss-Seidel style) :

$$x_i^{(k+1)} = \arg \min_t f\left(x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}, t, x_{i+1}^{(k)}, \dots, x_d^{(k)}\right)$$

Gradient par coordonnées (Jacobi style) :

$$x_i^{(k+1)} = \arg \min_t f\left(x_1^{(k)}, \dots, x_{i-1}^{(k)}, t, x_{i+1}^{(k)}, \dots, x_d^{(k)}\right)$$

La première utilise les valeurs fraîches, la seconde garde tout à l'itération k .

Conséquence : CD converge généralement plus vite car elle exploite immédiatement les améliorations.

3.5.3. Stratégies de parcours (Optionnel)

Cyclique : $i = 1, 2, \dots, d, 1, 2, \dots$ (ordre fixe)

Aléatoire : Tirer i uniformément dans $\{1, \dots, d\}$ à chaque itération

Greedy : Choisir $i = \arg \max_j \left| \frac{\partial f}{\partial x_j}(x) \right|$ (coûteux)

3.5.4. Convergence

Théorème 3.16

Si f est convexe et différentiable avec gradient L -Lipschitz :

$$\mathbb{E}[f(x^{(k)})] - f(x^*) = \mathcal{O}\left(\frac{d}{k}\right) \quad \text{pour CD aléatoire}$$

Interprétation : Facteur d car on ne met à jour qu'1 coordonnée sur d , mais coût par itération d fois plus faible.

3.5.5. Descente par blocs

Généralisation : mettre à jour plusieurs coordonnées simultanément.

Algorithme 3.17 (Block Coordinate Descent)

Partitionner : $\{1, \dots, d\} = B_1 \cup \dots \cup B_m$

À chaque itération, choisir un bloc B_j et minimiser selon toutes ses coordonnées :

$$x_{B_j} \leftarrow \arg \min_{u \in \mathbb{R}^{|B_j|}} f(x_1, \dots, x_{i_j-1}, u, x_{i_j+|B_j|}, \dots, x_d)$$

Cas d'usage : Variables naturellement groupées, parallélisation.

3.5.6. Avantages et limitations

Avantages

- ✓ Coût par itération très faible : $\mathcal{O}(n)$ au lieu de $\mathcal{O}(nd)$
- ✓ Solution exacte possible dans certains cas (Lasso)
- ✓ Pas de paramètre de pas à régler (minimisation exacte)
- ✓ Excellente performance pour problèmes structurés

Limitations

- × Convergence lente si variables fortement couplées
- × Nécessite une structure exploitable
- × Facteur d théorique dans la vitesse de convergence

3.5.7. Comparaison récapitulative

Méthode	Coût/iter	Variables	Conver- gence
GD	$\mathcal{O}(nd)$	Toutes	$\mathcal{O}\left(\frac{1}{k}\right)$
SGD	$\mathcal{O}(d)$	Toutes	$\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$
CD	$\mathcal{O}(n)$	Une	$\mathcal{O}\left(\frac{d}{k}\right)$

Remarque : SGD parcourt les **exemples**, CD parcourt les **variables**. Complémentaires !

Exercice

1. Pour $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ avec A diagonale, montrez que CD cyclique converge en une époque.
2. Implémentez CD pour Ridge et comparez avec la descente de gradient classique.

3.6. Méthode de Newton-Raphson

3.6.1. Motivation par développement de Taylor

La descente de gradient utilise uniquement l'approximation d'ordre 1. Que se passe-t-il si on utilise l'ordre 2 ?

Développement de Taylor à l'ordre 2

Si f est deux fois différentiable, pour un déplacement d :

$$f(x + v) \approx f(x) + \nabla f(x)^\top v + \frac{1}{2}v^\top H_f(x)v$$

où $H_f(x)$ est la matrice hessienne.

Idée : Minimiser cette approximation quadratique pour trouver la meilleure direction de descente.

3.6.2. Dérivation de l'algorithme

Pour minimiser l'approximation quadratique en v , on annule son gradient par rapport à v :

$$\nabla_v \left[f(x) + \nabla f(x)^\top v + \frac{1}{2}v^\top H_f(x)v \right] = \nabla f(x) + H_f(x)v = 0$$

Si $H_f(x)$ est inversible, on obtient :

$$v^* = -H_f(x)^{-1} \nabla f(x)$$

Algorithme 3.13 (Méthode de Newton)

Soit $f : \mathbb{R}^d \rightarrow \mathbb{R}$ deux fois différentiable. À partir d'un point initial $x^{(0)}$, on itère :

$$x^{(k+1)} = x^{(k)} - H_f(x^{(k)})^{-1} \nabla f(x^{(k)})$$

Hypothèses nécessaires :

- f est de classe $\mathcal{C}^2$
- $H_f(x^{(k)})$ est inversible à chaque itération

3.6.3. Propriétés de convergence

Théorème 3.14 (Convergence quadratique)

Si f est deux fois continûment différentiable et si :

- x^* est un minimum local avec $\nabla f(x^*) = 0$
- $H_f(x^*)$ est définie positive
- $x^{(0)}$ est suffisamment proche de x^*

Alors la méthode de Newton converge **quadratiquement** :

$$\|x^{(k+1)} - x^*\| \leq C \|x^{(k)} - x^*\|^2$$

pour une constante $C > 0$.

Interprétation de la convergence quadratique

La convergence quadratique signifie que l'erreur à l'itération $k + 1$ est proportionnelle au carré de l'erreur à l'itération k .

Exemple : Le nombre de chiffres corrects **double** à chaque itération !

Si $\|x^{(k)} - x^*\| = 10^{-m}$ (erreur avec m chiffres corrects), alors :

$$\|x^{(k+1)} - x^*\| \leq C \cdot (10^{-m})^2 = C \cdot 10^{-2m}$$

Donc environ $2m$ chiffres corrects (si $C \approx 1$).

Exemple 3.14.1 (Vitesse de convergence)

Supposons $C = 1$ et $\|x^{(0)} - x^*\| = 10^{-1} = 0.1$:

Itération k	Erreur $\ x^{(k)} - x^*\ $	Chiffres corrects
0	$10^{-1} = 0.1$	1
1	$10^{-2} = 0.01$	2
2	$10^{-4} = 0.0001$	4
3	$10^{-8} \approx 10^{-8}$	8
4	$10^{-16} \approx 10^{-16}$	16

En seulement 4-5 itérations, on atteint la précision machine ($\approx 10^{-16}$) !

Comparaison avec la descente de gradient :

Convergence linéaire (GD)

Pour la descente de gradient, on a typiquement :

$$\|x^{(k+1)} - x^*\| \leq \rho \|x^{(k)} - x^*\|$$

avec $\rho \in (0, 1)$ (par exemple $\rho = 0.9$).

L'erreur est multipliée par un facteur constant < 1 à chaque itération.

Exemple avec $\rho = 0.9$:

- Après 10 itérations :

$$\|x^{(10)} - x^*\| \approx 0.9^{10} \|x^{(0)} - x^*\| \approx 0.35 \|x^{(0)} - x^*\|$$

- Après 50 itérations :

$$\|x^{(50)} - x^*\| \approx 0.9^{50} \|x^{(0)} - x^*\| \approx 0.005 \|x^{(0)} - x^*\|$$

Beaucoup plus lent que la convergence quadratique !

Remarque : La convergence quadratique est **beaucoup plus rapide** que la convergence linéaire de la descente de gradient, mais seulement **localement** (près du minimum).

Remarque 3.14.2 (Convergence locale vs globale)

La méthode de Newton a une convergence **locale** :

- ✓ Très rapide **près** du minimum (convergence quadratique)
- ✗ Peut diverger si $x^{(0)}$ est loin de x^*
- ✗ Nécessite que $H_f(x^{(k)})$ soit définie positive

En pratique, on combine souvent :

1. Descente de gradient au début (convergence globale)
2. Newton à la fin (vitesse quadratique)

3.6.4. Avantages et inconvénients

Avantages

- Convergence quadratique (très rapide près du minimum)
- Pas de paramètre de pas à régler
- Direction de descente optimale pour l'approximation quadratique

Inconvénients

- Coût élevé : calcul et inversion de $H_f(x)$ en $\mathcal{O}(d^3)$
- Nécessite le stockage de la hessienne : $\mathcal{O}(d^2)$ en mémoire
- Convergence **locale** uniquement (nécessite un bon point initial)
- Peut diverger si $H_f(x)$ n'est pas définie positive

3.6.5. Exemple : régression linéaire

Exemple 3.15 (Newton pour les moindres carrés)

Pour $f(w) = \frac{1}{2n} \|y - Xw\|^2$:

- $\nabla f(w) = \frac{1}{n} X^\top (Xw - y)$
- $H_f(w) = \frac{1}{n} X^\top X$ (constante !)

La méthode de Newton donne :

$$\begin{aligned}
w^{(k+1)} &= w^{(k)} - \left(\frac{1}{n} X^\top X \right)^{-1} \cdot \frac{1}{n} X^\top (X w^{(k)} - y) \\
&= w^{(k)} - (X^\top X)^{-1} X^\top (X w^{(k)} - y)
\end{aligned}$$

Convergence en une seule itération vers $w^* = (X^\top X)^{-1} X^\top y$!

Explication : La fonction est quadratique, donc l'approximation d'ordre 2 est exacte.

Exercice

1. Montrez que pour une fonction quadratique $f(x) = \frac{1}{2} x^\top A x - b^\top x$, la méthode de Newton converge en une itération.
2. Calculez explicitement $x^{(k+1)}$ pour $f(x, y) = x^2 + 4y^2$ en partant de $(x^{(0)}, y^{(0)}) = (1, 1)$.
3. Comparez (théoriquement) le nombre d'opérations pour une itération de :
 - Descente de gradient
 - Méthode de Newton

3.7. Synthèse

Points clés :

- La descente de gradient suit la direction de plus forte descente locale (ordre 1)
- Le développement de Taylor justifie la décroissance pour un pas assez petit
- Le momentum accélère la convergence en accumulant les gradients passés
- La méthode de Newton utilise l'information d'ordre 2 (hessienne) pour une convergence quadratique
- Newton est optimal pour les fonctions quadratiques mais coûteux en grande dimension
- Les méthodes quasi-Newton (BFGS, L-BFGS) approximent la hessienne pour réduire le coût (je mets ça en plus)
- SGD échange la variance contre la vitesse : même gradient en espérance, mais fluctuations
- Mini-batch offre un compromis variance/vitesse optimal en pratique

Méthode	Ordre	Coût/iter	Convergence
GD	1	$\mathcal{O}(nd)$	$\mathcal{O}\left(\frac{1}{k}\right)$
SGD	1	$\mathcal{O}(d)$	$\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$
Momentum	1	$\mathcal{O}(nd)$	$\mathcal{O}\left(\frac{1}{k^2}\right)$ (Nesterov)

Newton	2	$\mathcal{O}(nd^3)$	Quadratique
BFGS	2 approx	$\mathcal{O}(nd^2)$	Super-linéaire
L-BFGS	2 approx	$\mathcal{O}(nd)$	Super-linéaire

En pratique :

- Pour les petits problèmes ($d < 1000$) : Newton ou BFGS
- Pour les problèmes moyens ($d < 10^6$) : L-BFGS
- Pour le deep learning ($d > 10^6$, $n > 10^6$) : SGD/mini-batch avec momentum (Adam, etc.)